



LESSON 09: NESTED CONTROL CONSTRUCTS

Loops Within Loops, Power Within Power

Module: Advanced Control Flow

Neon City Cyber Academy

Junior Agent Training Protocol



Your Team Today

Emily Chen - Handler

"Nested loops are where programming gets really powerful. Think matrices, grids, combinations - the real stuff."

Cole Martinez - Tech Lead

"Warning: nested loops multiply complexity fast. With great power comes great responsibility... and $O(n^2)$ time complexity!"



Quick Review: Single Loops

We've mastered:

- `while` and `for` loops
- `break` and `continue`
- `range()` variations
- Loop patterns (accumulator, counter, flag)

Today: Put loops INSIDE other loops!

What Are Nested Loops?

Definition: A loop inside another loop.

```
for i in range(3):  
    for j in range(3):  
        print(f"({i}, {j})")
```

Output:

```
(0, 0) (0, 1) (0, 2)  
(1, 0) (1, 1) (1, 2)  
(2, 0) (2, 1) (2, 2)
```



How Nested Loops Work

Execution pattern:

1. Outer loop runs **once**
2. Inner loop runs **completely**
3. Back to outer loop
4. Inner loop runs **completely again**
5. Repeat until outer loop finishes

Think: For every lap around the track (outer), you do 10 push-ups (inner).



Emily Explains: The Iteration Count

"Here's the math:

- Outer loop: 3 iterations
- Inner loop: 3 iterations per outer
- **Total: $3 \times 3 = 9$ iterations**

If both loops ran 100 times?

$100 \times 100 = 10,000$ iterations!

This is why nested loops can be slow!"



Visualization: Multiplication Table

```
for row in range(1, 6):  
    for col in range(1, 6):  
        product = row * col  
        print(f"{product:3}", end=" ")  
    print() # New line after each row
```

Output:

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25



Example: Drawing Patterns

Task: Print a grid of stars

```
rows = 4
cols = 5

for i in range(rows):
    for j in range(cols):
        print("*", end="")
    print() # New line
```

Output:

```
*****
*****
*****
*****
```



Cole's Pro Tip: Naming Loop Variables

"Use descriptive names for nested loops:



```
for row in range(...): for col in range(...)
```



```
for i in range(...): for j in range(...)
```



```
for i in range(...): for i in range(...) # WRONG!
```

NEVER reuse the same variable name in nested loops - you'll overwrite the outer loop variable!"

Challenge #1: Right Triangle

Mission: Print a right triangle pattern

```
*  
**  
***  
****  
*****
```

Hint: The number of stars equals the row number!

```
# Your code here
```



Solution: Right Triangle

```
for row in range(1, 6):  
    for col in range(row):  
        print("*", end="")  
    print()
```

Key insight: Inner loop runs `row` times!

- Row 1: 1 star
- Row 2: 2 stars
- Row 3: 3 stars

12
34

Nested Loops with Conditionals

Combining powers:

```
for i in range(1, 6):  
    for j in range(1, 6):  
        if i == j:  
            print("X", end=" ")  
        else:  
            print("0", end=" ")  
    print()
```

Output (diagonal pattern):

```
X 0 0 0 0  
0 X 0 0 0  
0 0 X 0 0  
0 0 0 X 0  
0 0 0 0 X
```

Emily's Pattern Recognition

"This is a **diagonal** pattern - where row equals column.

In mathematics, this is the **identity matrix**!

You're learning computer science AND linear algebra. Pretty cool, right?"

Example: Prime Number Sieve

Finding all primes from 2 to 50:

```
for num in range(2, 51):
    is_prime = True

    for divisor in range(2, num):
        if num % divisor == 0:
            is_prime = False
            break

    if is_prime:
        print(num, end=" ")
```



Time Complexity: $O(n^2)$

Big O Notation measures algorithm efficiency.

Single loop: $O(n)$ - linear time

```
for i in range(n): # n iterations
    print(i)
```

Nested loop: $O(n^2)$ - quadratic time

```
for i in range(n): # n iterations
    for j in range(n): # n iterations each
        print(i, j) # Total: n x n
```

Cole's Performance Warning

" $O(n^2)$ algorithms get slow FAST:

- $n = 10 \rightarrow 100$ operations 
- $n = 100 \rightarrow 10,000$ operations 
- $n = 1,000 \rightarrow 1,000,000$ operations 
- $n = 10,000 \rightarrow 100,000,000$ operations 

Always ask: can I avoid nested loops?"

Challenge #2: Coordinate Grid

Mission: Print all coordinate pairs from (0,0) to (3,3)

Example output:

```
(0,0) (0,1) (0,2) (0,3)
(1,0) (1,1) (1,2) (1,3)
(2,0) (2,1) (2,2) (2,3)
(3,0) (3,1) (3,2) (3,3)
```



Solution: Coordinate Grid

```
for x in range(4):  
    for y in range(4):  
        print(f"({x},{y})", end=" ")  
    print() # New line after each row
```

This is a 2D grid! Used in:

- Games (chess boards, maps)
- Images (pixels)
- Data tables



Nested While Loops

Yes, you can nest while loops too!

```
outer = 0
while outer < 3:
    inner = 0
    while inner < 3:
        print(f"[{outer},{inner}]", end=" ")
        inner += 1
    print()
    outer += 1
```

Same output as nested for loops!

Breaking Out of Nested Loops

Problem: `break` only exits the **innermost** loop!

```
for i in range(5):
    for j in range(5):
        if i == 2 and j == 2:
            break # Only breaks inner loop!
        print(f"({i},{j})", end=" ")
    print()
```

What if you want to exit both loops?



Solution: Flag Variable

```
found = False

for i in range(5):
    for j in range(5):
        if i == 2 and j == 2:
            found = True
            break
    if found:
        break
# Rest of code
```

Or: Put the nested loops in a function and use `return` !

Emily's Advanced Technique

"In Python, there's another trick using `else` with loops:

```
for i in range(5):
    for j in range(5):
        if condition:
            break
    else:
        continue # Only if inner didn't break
break # Only if inner did break
```

But the flag method is clearer for beginners!"

Challenge #3: Find Pair Sum

Mission: Find two numbers in range 1-10 that add up to 13.

Print the first pair you find, then stop.

```
target = 13
# Your nested loop here
```



Solution: Find Pair Sum

```
target = 13
found = False

for i in range(1, 11):
    for j in range(1, 11):
        if i + j == target:
            print(f"Found: {i} + {j} = {target}")
            found = True
            break
    if found:
        break
```

Output: Found: 3 + 10 = 13 (or any valid pair)

Triple Nested Loops (3D!)

Warning: Now we're at $O(n^3)$!

```
for x in range(3):  
    for y in range(3):  
        for z in range(3):  
            print(f"({x},{y},{z})")
```

Total iterations: $3 \times 3 \times 3 = 27$

Used for: 3D graphics, volumetric data, game worlds



Cole's Complexity Warning

"Each nesting level multiplies the iterations:

- 1 level: $O(n) = 100$ ops
- 2 levels: $O(n^2) = 10,000$ ops
- 3 levels: $O(n^3) = 1,000,000$ ops
- 4 levels: $O(n^4) = 100,000,000$ ops

Rule: Avoid more than 2-3 levels of nesting!"



Real-World Application: Image Processing

```
# Process a 1920x1080 image
width = 1920
height = 1080

for y in range(height):
    for x in range(width):
        # Process pixel at (x, y)
        pixel = get_pixel(x, y)
        process_pixel(pixel)
```

Total pixels: $1920 \times 1080 = 2,073,600$

This is why image processing is computationally expensive!



Common Error: Incorrect Loop Bounds

Problem: Using the same range for different purposes

```
# Buggy code
for i in range(5):
    for j in range(i): # j depends on i!
        print("*", end="")
    print()
```

This creates a triangle, which might not be what you want!



Common Error: Forgetting Print()

```
for i in range(3):  
    for j in range(3):  
        print("*", end="")  
# Missing print() for new line!
```

Output: `*****` (all on one line!)

Fix: Add `print()` after inner loop.



Emily's Debugging Checklist for Nested Loops

"When nested loops misbehave:

1. ☒ Add debug prints to see which loop is running
2. ☒ Test with small numbers first (3×3, not 100×100)
3. ☒ Verify each loop variable updates correctly
4. ☒ Check loop bounds carefully
5. ☒ Draw what the output SHOULD look like
6. ☒ Count expected iterations (outer × inner)"



Final Challenge: Number Grid

Mission: Print a number grid like this:

```
1  2  3  4  5
2  4  6  8 10
3  6  9 12 15
4  8 12 16 20
5 10 15 20 25
```

Hint: Each cell is `row × col`



Solution: Number Grid

```
for row in range(1, 6):  
    for col in range(1, 6):  
        result = row * col  
        print(f"{result:3}", end=" ")  
    print()
```

Formatting tip: `{result:3}` pads to 3 characters for alignment!

🎓 Key Takeaways

- ✓ **Nested loops** run one loop inside another
- ✓ **Iterations multiply:** $\text{outer} \times \text{inner}$
- ✓ **$O(n^2)$ complexity** - gets slow with large n
- ✓ **Use flag variables** to break from multiple loops
- ✓ **Limit nesting** to 2-3 levels max
- ✓ **Perfect for grids, matrices, combinations**

Next Mission: Lesson 10

Coming Up:

- Functions – reusable code blocks!
- `def` keyword and function structure
- DRY principle (Don't Repeat Yourself)
- Stack frames and function calls

Agent Status: Nested Structures MASTERED 



Mission Stats

Concept: Nested Control Structures

Difficulty: ★★★★★

Skills Unlocked:

- 2D/3D iteration
- Time complexity analysis
- Pattern generation
- Algorithm optimization

Next Level: Functions & Abstraction

